

Projet – Recherche et localisation de source

Mission 1 :

Nous disposons du fichier Robot.py qui fournit le modèle cinématique du robot, le gestionnaire de waypoints et la classe qui nous permet d'enregistrer l'évolution de notre robot. On a aussi la classe Potential qui nous permet d'obtenir la concentration du polluant avec la position du robot. Et pour finir, le Timer qui nous permet de gérer la mise à jour des consignes d'orientation et de vitesse.

L'objectif de cette mission est d'ajouter à ces codes une estimation du gradient et la logique pour détecter la source lorsque le robot se dirige vers le niveau maximum de polluant.

Pour ce faire nous allons utiliser une boucle qui a le comportement suivant :

- A chaque période de la boucle de position, on mesure le potentiel et on mémorise la position. On stocke tout dans notre historique.
- On modifie ou non notre variable qui garde la plus grosse valeur (du niveau de pollution) et sa position. Cette donnée sert pour déterminer la source.
- A partir de nos dernières mesures, on évalue le gradient et on filtre le résultat pour stabiliser la direction.
- On ajuste notre vitesse et notre orientation en fonction des résultats précédents.
- On ajoute des critères d'arrêt pour que le robot se stoppe quand il a trouvé ce qu'il cherche.

Après la simulation, le script sauvegarde :

- L'historique des mesures avec la position de la mesure et le taux de pollution
- La durée, le niveau max de pollution et la position (estimée) de la source.
- La trajectoire du robot pour l'exploration.
- La série temporelle des potentiels.
- Le nuage 3D (x, y, potentiel) qui illustre le relevé de niveau.

Les paramètres principaux sont fixés de sorte que la simulation soit réelle. La vitesse de déplacement, la vitesse de rotation, etc... Le cahier des charges a été établi lors de la lecture du projet :

- Le gain pour transformer la norme du gradient en vitesse linéaire.
- La vitesse de déplacement maximum.
- La vitesse de rotation maximum.
- Le coefficient du filtre sur le gradient.
- Le seuil pour lequel on déclenche une exploration plus précise (plus lente)
- Le rayon du disque autour de la meilleure mesure (pour la recherche)

Voici le code qui a été implémenté :

```
# loop on simulation time
for t in simu.t:
    # position control loop
    if timerPositionCtrl.isEllapsed(t):
        pos_act = np.array([robot.x, robot.y])
        pot_val = pot.value(pos_act)
        mesure.append((t, pos_act[0], pos_act[1], pot_val))

        if pot_val > max_mesure["value"]:
            max_mesure = {"position": pos_act.copy(), "value": pot_val}

        if len(mesure) >= 2:
            mesure_av = mesure[-2]
            pos_avt = np.array([mesure_av[1], mesure_av[2]])
            step = pos_act - pos_avt
            stepNormSq = float(np.dot(step, step))
            if stepNormSq > 1e-9:
                grad_mesure = ((pot_val - mesure_av[3]) / stepNormSq) * step
                grad_estime = grad_filtre * grad_estime + (1.0 - grad_filtre) * grad_mesure

            gradNorm = float(np.linalg.norm(grad_estime))
            step_explo += 1

            if (gradNorm < minGradientNorm) or (step_explo >= explorationRefresh):
                explorationSeed += random.uniform(-math.pi / 4.0, math.pi / 4.0)
                headingRef = wrap_to_pi(explorationSeed)
                step_explo = 0
                stepAmplitude = explorationstep
            else:
                headingRef = math.atan2(grad_estime[1], grad_estime[0])
                stepAmplitude = explorationstep + min(0.5 * gradNorm, 1.0)

        cible = pos_act + stepAmplitude * np.array([math.cos(headingRef), math.sin(headingRef)])

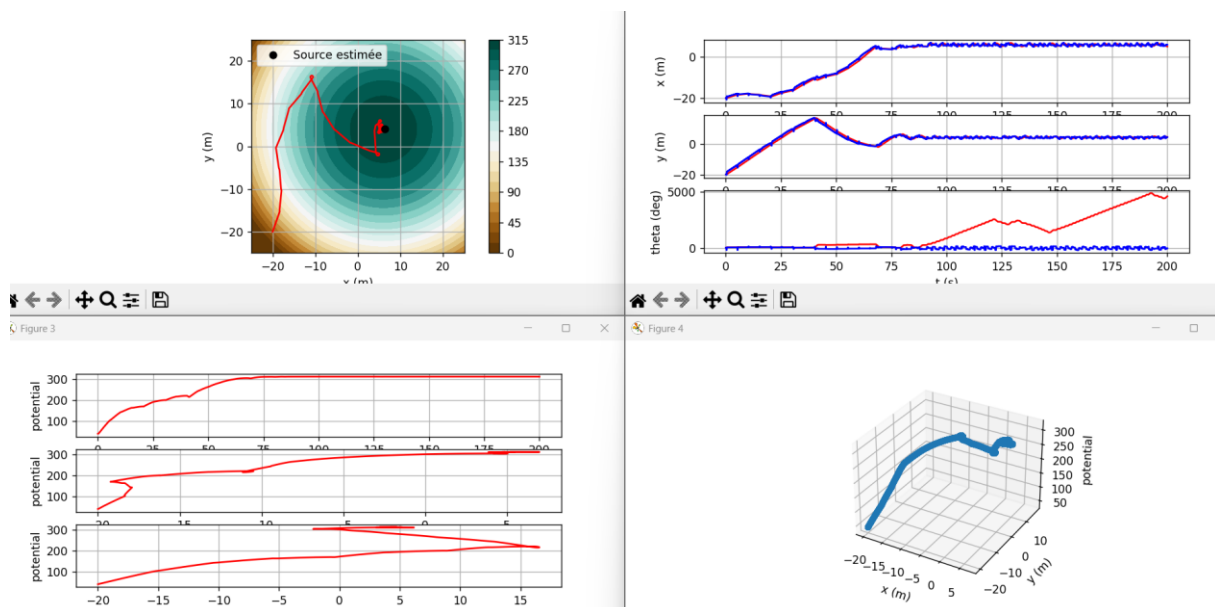
    # update the waypoint manager with the freshly computed objective
    WPManager.currentWP = [cible[0], cible[1]]
    WPManager.xr = cible[0]
    WPManager.yr = cible[1]

    dx = WPManager.xr - robot.x
    dy = WPManager.yr - robot.y
    dist = float(np.hypot(dx, dy))

    Vr = kpPos * dist
    if dist < stopDistance:
        Vr = 0.0
    Vr = max(0.0, min(Vr, Vmax))

    thetar = math.atan2(dy, dx)
    thetar = wrap_to_pi(thetar)
```

Voici les résultats après simulation :



Voici ce que les résultats nous montrent :

Le robot démarre depuis la position initiale fixée. A chaque période de la boucle de position, le robot mesure la concentration et l'enregistre dans l'historique. On ajoute des nouveaux waypoints pour guider le robot et il se dirige en direction de ces waypoints. En fin de recherche, on affiche la meilleure mesure ainsi que sa position estimée et le temps de recherche. Et on trace le chemin du robot. Les résultats montrent que le robot quitte la zone de départ et s'oriente vers la source en corrigeant sa direction en fonction de ce qu'il a trouvé. A la fin il se stabilise vers le centre et s'arrête lorsqu'il pense avoir trouvé le point d'émission. La durée de recherche est de 120 minutes (on suppose 1 seconde en simulateur = 1 minute temp réel). On pourrait améliorer cette durée en faisant varier quelques paramètres pour répondre aux contraintes d'autonomie (batterie du robot) et au cahier des charges.

Nous avons pour le moment traité le cas le plus simple avec un seul point d'émission. Nous allons maintenant augmenter la difficulté et ajouter différents centres d'émissions. On reste en génération random False pour effectuer nos tests et une fois nos algorithmes finis, nous réactiverons ce paramètre.

Après avoir testé notre code sur la carte du niveau 2, on se rend compte que le robot n'arrive pas à s'orienter correctement, il se « coince » lui-même et tourne en rond autour du centre d'émissions. Malgré plusieurs correctif et la modification de tous les paramètres susceptibles de provoquer ce problème, aucune simulation n'a correctement été réussi. Afin de gagner du temps, nous repartons de zéro pour adopter une nouvelle logique.

Avant d'implémenter notre nouvelle stratégie de déplacement, nous ajoutons un menu de sélection pour choisir facilement à chaque exécution notre niveau et le type de génération.

```
# saisie utilisateur
choix_difficulte = int(input("Choisir la difficulté (1, 2 ou 3) : "))
choix_hasard = input("Nuage aléatoire ? (T/F) : ").strip().upper() == 'T'

# potential
pot = Potential.Potential(difficulty=choix_difficulte, random=choix_hasard)
```

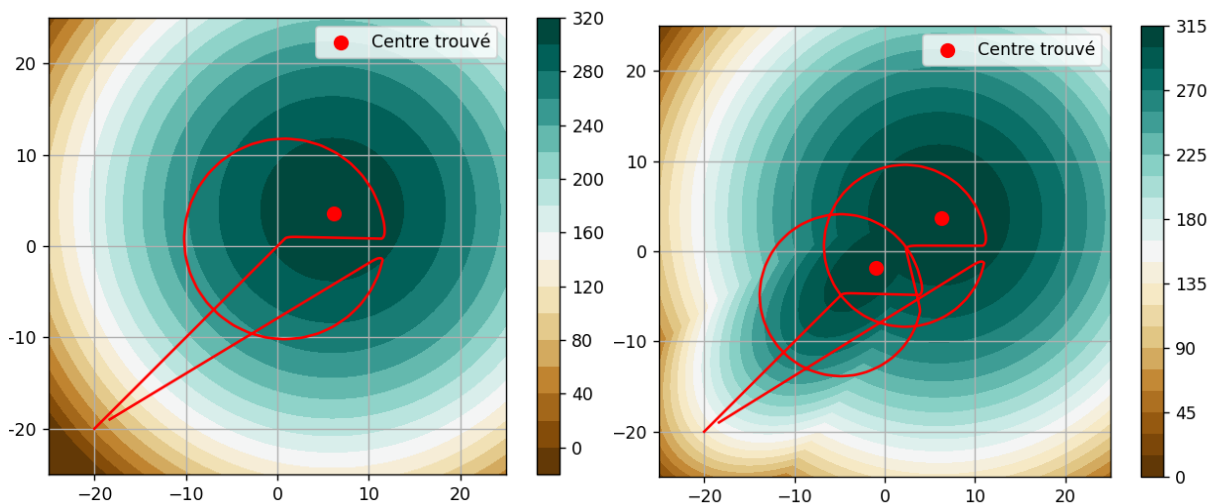
Nous pouvons désormais coder la nouvelle stratégie. Voici la logique :

- Le robot s'oriente vers le centre de la zone et commence à faire des mesures à intervalle de temps réguliers.
- On compare cette mesure à une seuil de détection (ici 305).
- Si la mesure est plus faible, on continu d'avancer tout droit.
- Si la mesure est plus élevée on passe en mode recherche.

Le mode recherche consiste à effectuer un cercle autour du point d'activation. Durant le cercle on fait des mesures à intervalle de temps régulier. Une fois le cercle fini, on exploite les mesures effectuées pendant le cercle. Si on a une mesure qui est supérieur au seuil alors on évalue la différence entre les deux mesures et on fait une estimation du centre qui doit normalement se trouver entre ces deux mesure (sur la droite reliant ces deux mesures). En fonction du niveau de difficulté et du nombre de noyau trouvé, une fois l'estimation faite soit on retourne au point de départ (on a trouvé tous nos noyaux) soit on va refaire un cercle de détection à ce point (si nous n'avons pas assez de noyau, on effectue un cercle pour confirmer qu'il n'y a pas un autre noyau proche du premier). On prend la valeur max des mesures du cercle.

Une fois le nombre de noyau trouvé (la tâche terminée), on ajoute un retour au point de départ car le robot ne doit pas rester à l'endroit où le niveau de polluant est le plus élevé sinon personne ne pourrait venir le chercher.

Voici les résultats obtenus avec cette nouvelle logique de navigation :



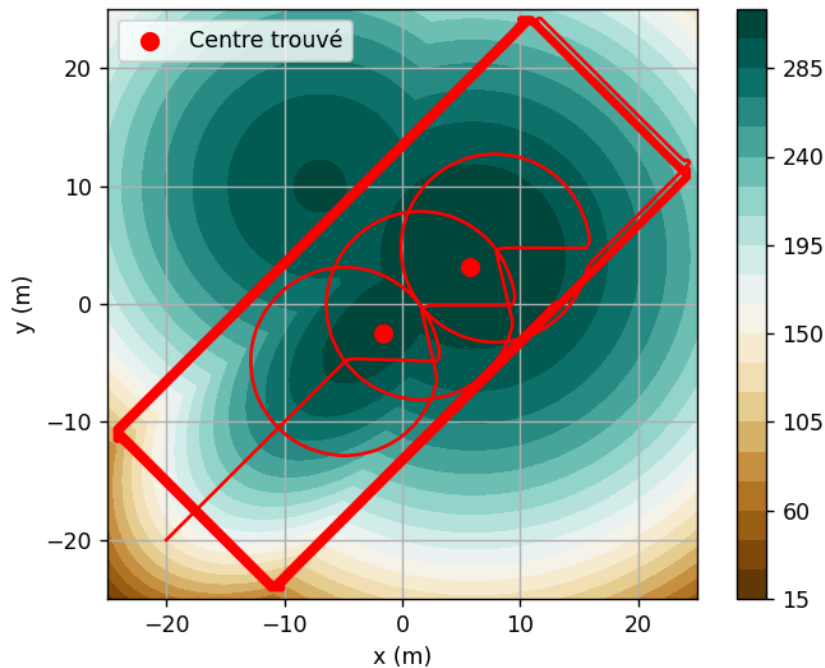
Pour les niveaux de difficulté 1 et 2, on constate que le robot arrive à détecter les noyaux d'émissions plutôt précisément, avec un retour propre au point initial. En réalité ces résultats sont plutôt précis pour 2 raisons :

- Nous sommes en mode fixe (génération random = False), les noyaux sont placés de manière simple et au centre de la carte.
- Comme nous avons tout le temps la même disposition, nous avons ajustés les paramètres (rayon du cercle de détection, distance de fusion) afin qu'elle fonctionne parfaitement avec notre situation.

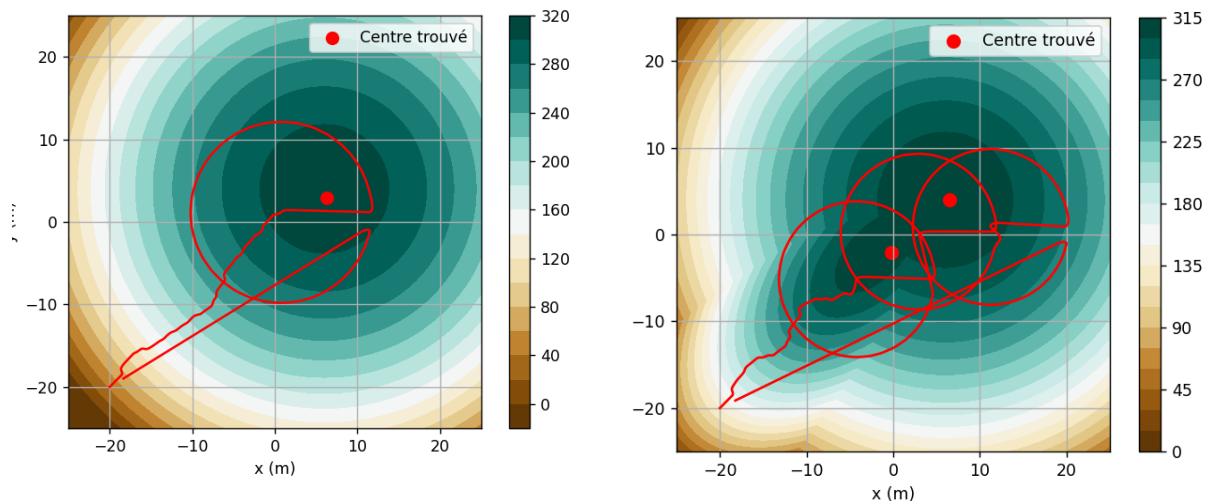
Cependant, en mode de génération random, la taille des zones et leurs positions varient. Pour le moment nous nous concentrons uniquement sur la logique, nous verrons par la suite pour ajuster ces paramètres et les rendre dynamique car pour le moment, notre plus gros problème est sur le niveau de difficulté 3.

Jusqu'à présent, le robot se contente d'avancer tout droit vers le centre et espère tomber sur la zone. Cependant pour le niveau 3, ce ne sera pas le cas même si on peut voir qu'il passe proche de notre 3-ème noyau, la mesure effectuée à ce point n'est pas assez élevée pour déclencher notre cercle d'analyse. Il faudrait donc implémenter une nouvelle logique de navigation.

Voici ce qu'il se passe pour le niveau 3 :

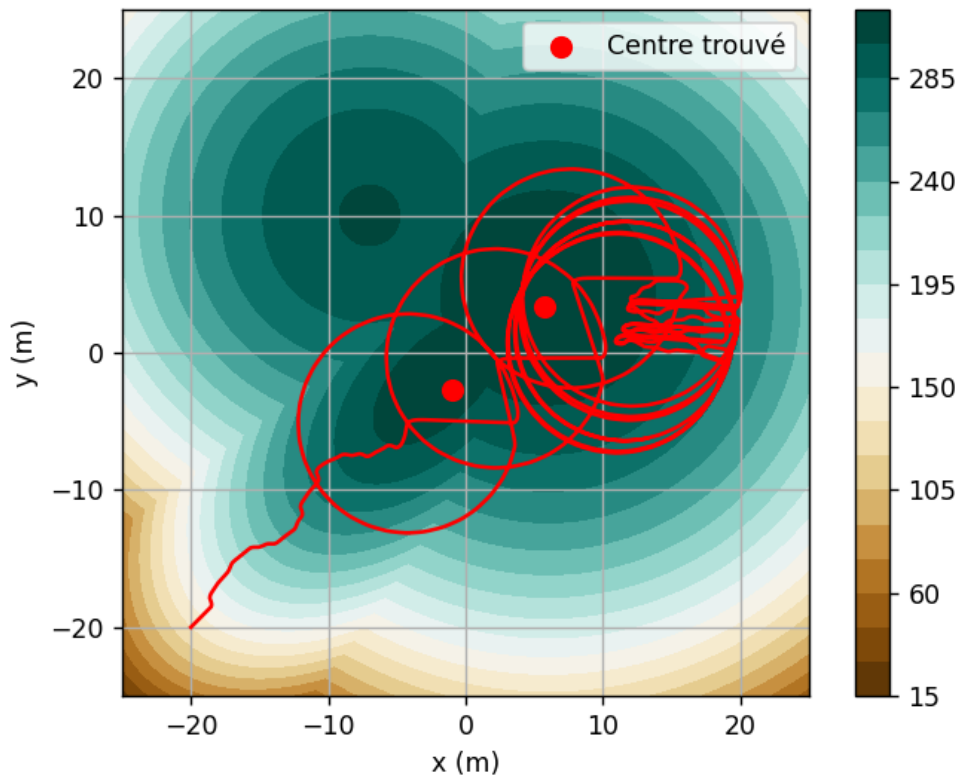


Voici la nouvelle logique de navigation : Le robot utilise les deux dernières mesures et la mesure actuelle pour déterminer dans quelle direction il est le plus probable que la source se trouve. Ainsi, il s'orientera de manière naturelle vers les centres, tout en gardant la même logique de détection.



Le résultat obtenu pour le niveau 1 et 2 est toujours correct, mais pas pour le niveau 3 encore un fois malgré nos changements.

Voici ce qu'il se passe :



Deux interprétations sont possibles :

- Une fois que le robot a fini ces cercles de détections, il reprend son mode de navigation mais en se déplaçant de manière intelligente, il retourne en direction de la même source d'émissions, raison pour laquelle il fait plein de cercle de détection, pour trouver un nouveau noyau mais il y en a déjà un trop proche (distance de fusion bloque la création d'un nouveau noyau).
- Il effectue des cercles de détection en boucle car il n'a pas trouvé assez de noyau et donc il continue à en chercher un avec les résultats de ces cercles de détection. Mais il reste bloqué en faisant des allers-retours entre les mêmes points (rayon fixe donc il retrouve plus ou moins le point l'ancien centre comme point max).

On va commencer par essayer de fixer la navigation dans un premier temps.

Jusqu'à présent, on stockait toutes nos mesures dans une simple liste. On va désormais changer ce fonctionnement pour stocker nos valeurs non pas dans une liste mais dans une matrice. Cela nous servira pour déterminer les « zones non explorées » et sera utile pour la Mission 2.

Comment fonctionne le remplissage de cette matrice :

- On initialise toutes nos valeurs à 0.
- Lorsqu'on fait une mesure, on arrondi les coordonnées x et y à l'entier le plus proche.
- On regarde la valeur actuelle de la case en $M[x][y]$, si cette valeur est 0 alors on remplace par notre mesure.
- Si la valeur de cette case n'est pas nulle, on fait la moyenne entre la valeur de la case et notre valeur et on remplace par le résultat obtenu.

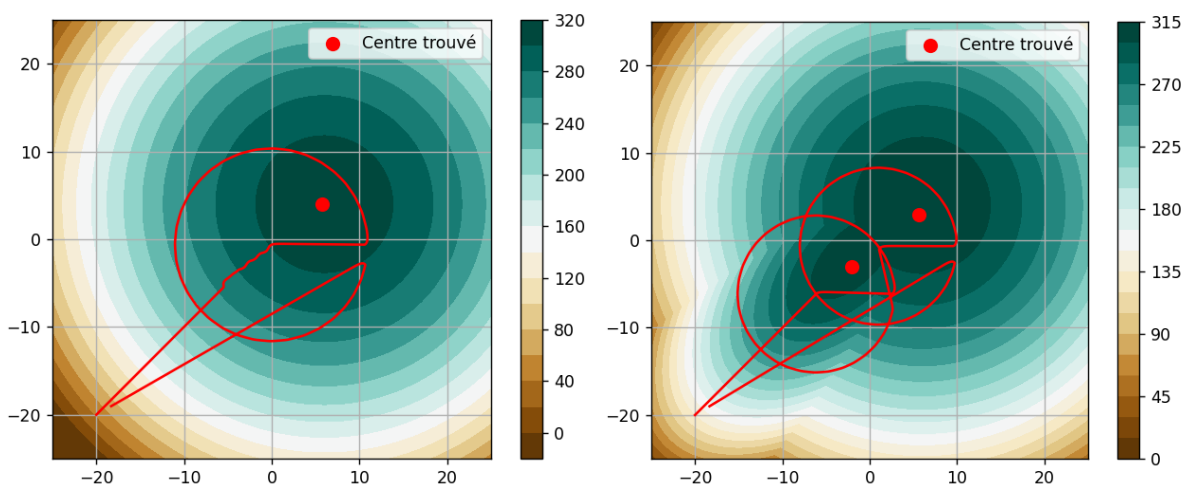
Ainsi, en parcourant notre matrice, on est désormais capable de déterminer les « zones non explorées » (carré avec que des 0).

On modifie le fonctionnement de notre navigation. Désormais on est capable de déterminer les zones à aller vérifier avec notre matrice :

- On parcourt notre matrice pour trouver la plus grosse zone non explorée.
- On détermine le centre de cette zone
- On demande au robot d'aller à cette position
- Le robot va directement à ce point et reprend un fonctionnement normal (cercle de détection, navigation, etc.) lorsqu'il se trouve à une certaine distance de ce point (défini par une variable)

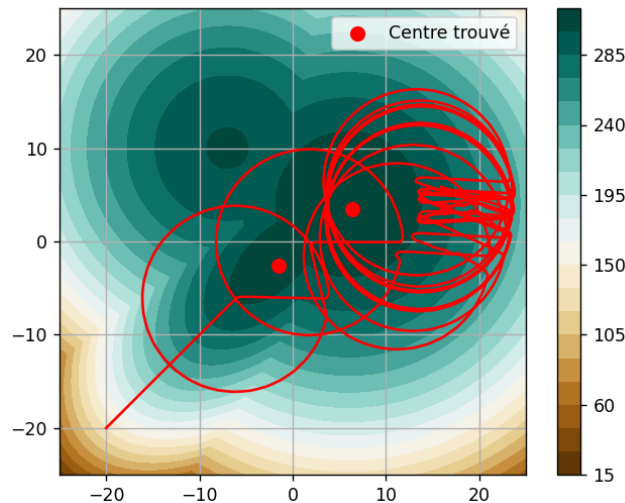
Ainsi, on est capable d'orienter le robot pour qu'il arrête de rester bloqué sur la même zone et qu'il aille découvrir de nouvelle partie de notre terrain où il pourrait potentiellement trouver des nouveaux noyaux.

Voici le résultat sur les difficultés 1 et 2 :



On constate qu'au début, le robot adopte une trajectoire directe, en direction du centre (matrice vide donc centre de la zone inexplorée est le centre du terrain). Et qu'une fois arrivé à une certaine distance de ce point, il commence à s'orienter de manière autonome.

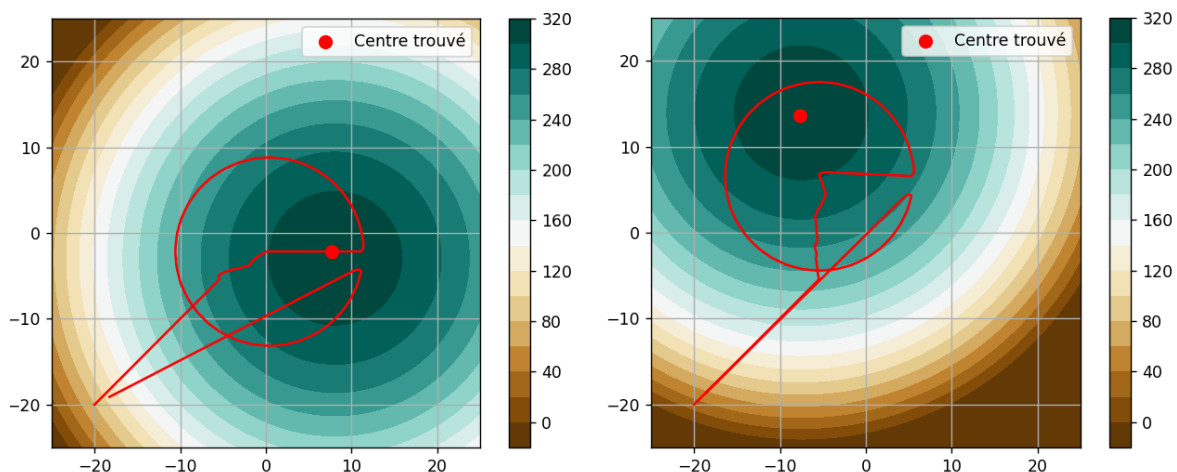
Cependant, encore une fois, pour la difficulté 3 ce mode de fonctionnement ne marche pas. Voici ce qu'on obtient :

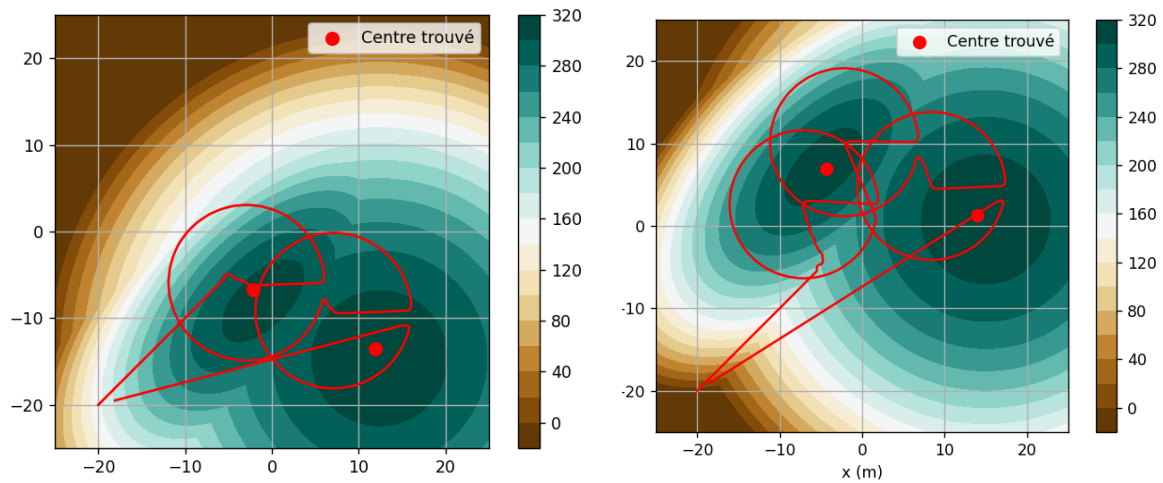


Nous avons corrigé un comportement que nous avons identifiés. Pour corriger le deuxième problème (effectue des cercles de détection en boucle), il faudrait ajouter un compteur qui compte combien de cercle de détection on fait d'affiler, si on dépasse se compteur alors on sort et on regarde la matrice pour aller dans une nouvelle zone.

Malgré cette implémentation, le système ne fonctionne toujours pas. Il reste bloqué à faire des cercles de détection jusqu'à la fin du temps de simulation. Par manque de temps, nous sommes contraints d'abandonner afin de pouvoir travailler sur la Mission 2.

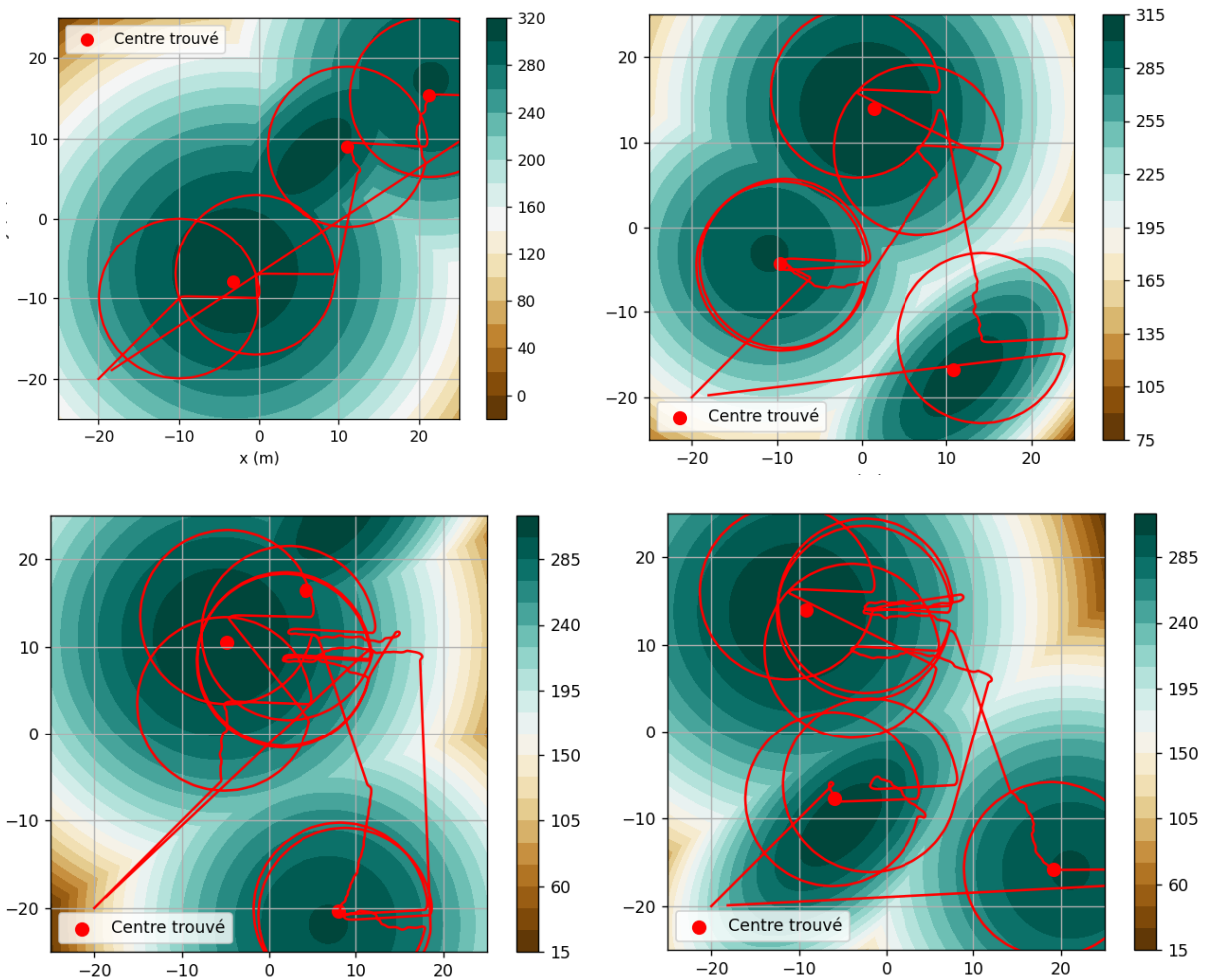
On décide tout de même d'essayer le code avec des génération en random pour voir le résultat obtenu avec d'autres nuages.



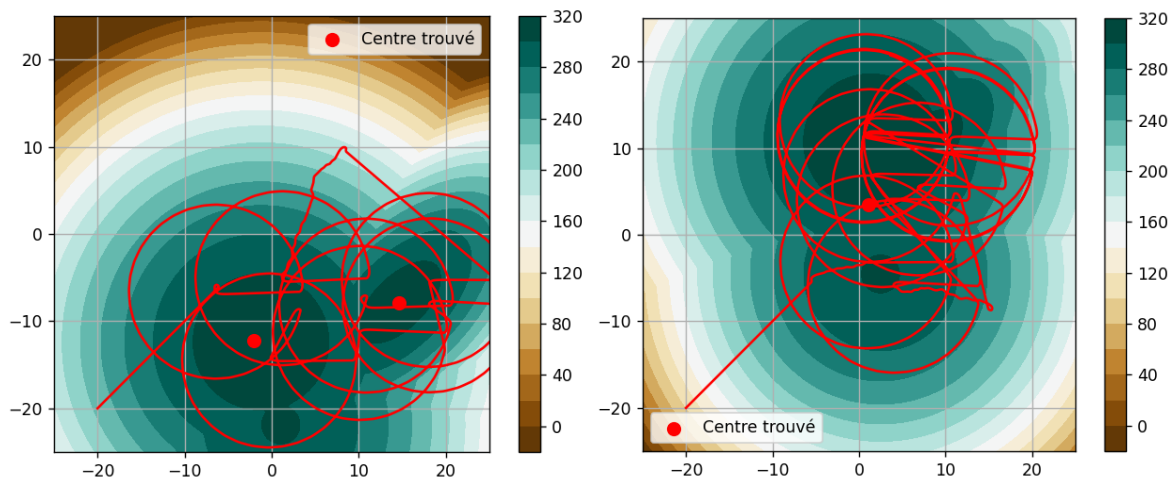


Pour les difficultés 1 et 2 tout fonctionne correctement, les centres sont trouvés de manière plutôt rapide et précise.

Pour la difficulté 3 on a tout de même de bon résultat surtout lorsque les zones sont bien espacées.



Par contre, quand les zones sont vraiment rapproché, les résultats sont catastrophique...



Mission 2 :

L'objectif est de déterminer la forme du nuage de pollution à partir des mesures que nous avons fait dans la Mission 1. On veut ici, créer une nouvelle carte avec une légende et des couleurs pour voir à quoi ressemble les nuages de pollution sur le terrain exploré.

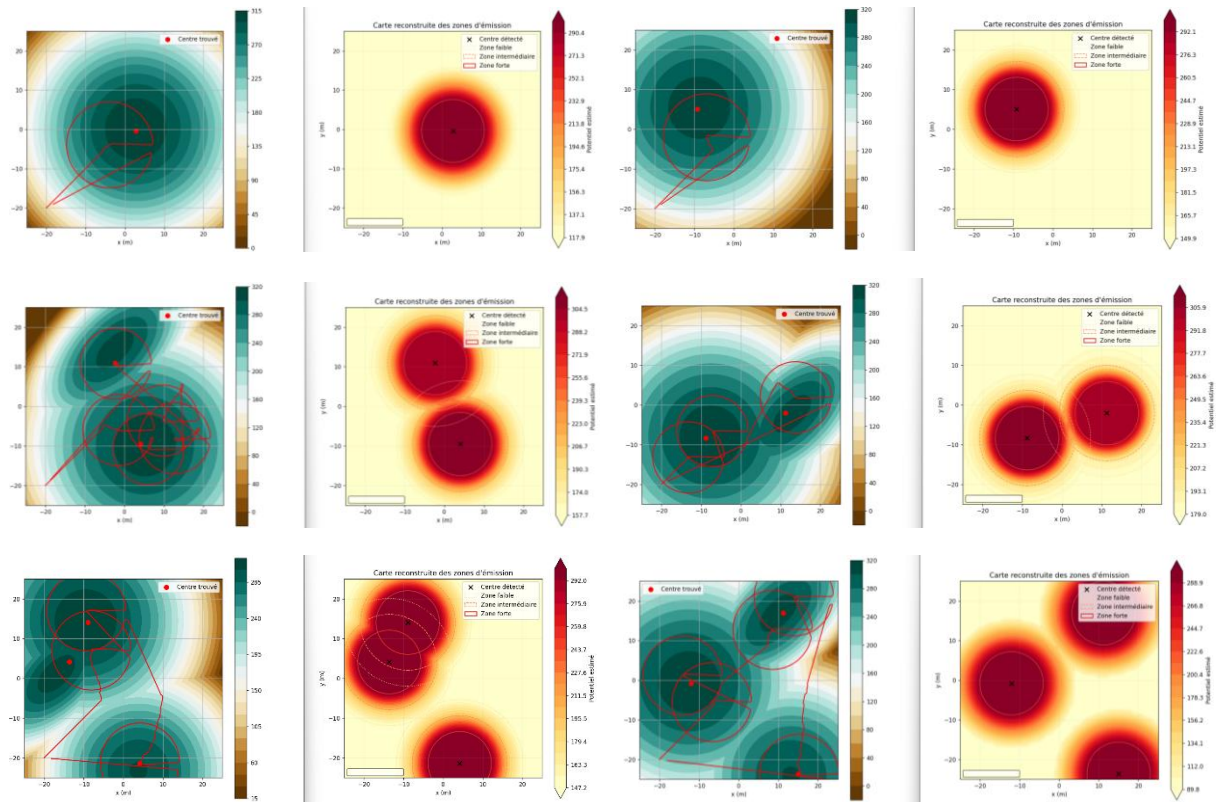
Pour ce faire voici ce que nous faisons :

- On dispose de toutes les mesures avec leurs positions stockées dans notre matrice.
- On dispose de nos centres (estimés)
- On grâce à nos cercles de détection, on est capable d'estimer la taille de la « zone forte » autour de notre centre.
- Ensuite, autour de ce point on diminue la concentration petit à petit de manière linéaire.
- On fait attention lorsque deux noyaux sont collés pour ne pas avoir de résultats non cohérents.

Voici le code qui a été implémenté pour gérer ces paramètres :

```
class ParametresCercles:
    seuil_ratio: float = 0.98
    pourc_dist: float = 70.0
    facteur_min_reference: float = 0.6
    rayon_min: float = 2.5
    largeur_ratio: float = 1
    largeur_min: float = 2.0
```

Voici les résultats après simulation :



Voici ce que les résultats nous montrent :

La représentation finale met en évidence que la concentration maximale est autour de nos noyaux d'émissions (estimés). La concentration réduit petit à petit plus on s'éloigne du centre de pollution.

Par soucis de temps, nous avons fait le choix de représenter nos nuages de pollutions uniquement avec des cercles. Nous avons essayé avec les ellipses mais les résultats n'étaient pas concluants, le principal problème était la taille et l'orientation de celles-ci.

Nous avons désormais une représentation du nuage de polluant estimé avec nos mesures qui est plutôt conforme à celui fournis.

Cependant, au début, nous avons précisé que cette simulation avait pour but de représenter un cas réel, avec des vitesses de déplacement et rotation adapté, etc... Les temps de simulations sont longs et le robot n'est pas forcément très efficace dans sa recherche. On pourrait améliorer certains paramètres pour le rendre encore plus rapide et plus précis afin de répondre à la contrainte d'autonomie du robot (batterie).